HEART DISEASE PREDICTION

End-to-End MLOps Solution

FINAL ASSIGNMENT REPORT

Field	Details
Student	Rahul Tiwari
Student ID	2025ae05122@wilp.bits-pilani.ac.in
Course	MLOPS
Institution	BITS
Submission date	20 October 2026
GitHub	Open GitHub repository
Docker image	Image: docker.io/2025ae05122/heart-disease-api:latest Repository: Open Docker Hub repository

Academic-use notice

This classifier is an educational MLOps demonstration and is not a validated medical diagnostic system.

1. Setup and Installation

The repository supports a clean Python setup, script-based training, automated tests, Docker packaging and local Kubernetes deployment.

Local Python setup

```
git clone https://github.com/2025ae05122/MLOPS-Assignment-1.git
cd MLOPS-Assignment-1
python -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

Dataset, training and tests

```
python data/download_dataset.py
python src/train.py
pytest -v
ruff check .
```

Container and local deployment

```
docker pull 2025ae05122/heart-disease-api:latest
minikube start
kubectl apply -f kubernetes/deployment.yaml
kubectl apply -f kubernetes/service.yaml
kubectl get deployments,pods,services
minikube service heart-disease-api --url
```

The service URL is temporary. On macOS with the Docker driver, the terminal running the Minikube service tunnel must remain open. The generated URL can be used with /health, /predict, /metrics and /docs.

Clean-setup success criteria

- Dependencies install without manual package additions.
- Training recreates the model and metadata artefacts.
- Tests and linting complete successfully.
- Minikube shows the deployment, two running pods and the service.
- POST /predict accepts sample/sample_request.json and returns prediction plus confidence.

2. End-to-End Architecture

The architecture separates reproducible model development, automated delivery and runtime observability. The same packaged sklearn pipeline is used for evaluation and API inference, reducing training-serving skew.

#	Architecture stage	Output / Responsibility
1	UCI Heart Disease Dataset	Raw CSV / download script
2	Data Validation, Cleaning and EDA	Missing values, histograms, heatmap, class balance
3	Reproducible Feature Pipeline	Median/mode imputation, one-hot encoding, scaling
4	Training and Selection	Logistic Regression + Random Forest, GridSearchCV, 5-fold CV
5	MLflow Tracking and Packaging	Parameters, metrics, plots, joblib pipeline, metadata
6	FastAPI and Docker	/health, /predict, /metrics; published Docker image
7	GitHub Actions CI/CD	Lint, train, test, build, endpoint verification, push
8	Minikube Deployment	2 replicas, LoadBalancer service, local tunnel
9	Observability	Structured request logs and Prometheus metrics

Design choices

- ColumnTransformer encapsulates numerical and categorical preprocessing.
- GridSearchCV and stratified five-fold CV make model selection repeatable.
- GitHub Actions acts as the integration gate before registry publication.
- Docker Hub provides the immutable deployable unit consumed by Minikube.
- FastAPI exposes explicit health, prediction and metrics contracts.
- Prometheus and structured logs provide operating evidence.

3. Data, EDA and Modelling Choices

The UCI Heart Disease dataset is converted to a binary target: 0 for absence and 1 for presence of heart disease. The executed notebook records 303 source rows, 13 original features, six missing values before handling and zero after preprocessing.

EDA performed

Visual	Purpose
Numerical histograms	Inspect feature distributions, skewness and possible outliers
Correlation heatmap	Inspect linear relationships among numerical variables
Target class bar chart	Check disease/no-disease class balance
Confusion matrix	Show final classification errors
ROC curve	Assess ranking behaviour across thresholds

Preprocessing and models

Area	Choice
Numerical	Median imputation + StandardScaler
Categorical	Most-frequent imputation + one-hot encoding
Split	Stratified train/test split: 242 training, 61 test records
Models	Logistic Regression and Random Forest
Tuning	GridSearchCV
Validation	Stratified five-fold cross-validation
Selection metric	Mean CV ROC-AUC

Logistic Regression provides an interpretable linear baseline, while Random Forest tests non-linear relationships and interactions. Both are tuned under the same validation strategy for a fair comparison.

4. Model Results and MLflow Tracking

Cross-validation comparison

Model	Accuracy	Precision	Recall	ROC-AUC	Best parameters
Logistic Regression	0.8427	0.8517	0.8008	0.9113	C=0.1; solver=liblinear
Random Forest	0.8139	0.8112	0.7826	0.8995	400 trees; max_depth=None; min_split=10

Held-out evaluation

Metric	Score
Accuracy	0.8689
Precision	0.8125
Recall	0.9286
ROC-AUC	0.9610

Logistic Regression was selected because it achieved the highest mean five-fold CV ROC-AUC. Evaluation was then performed on the untouched 61-record test set.

Experiment tracking summary

- MLflow records separate Logistic Regression and Random Forest runs.
- Model parameters and run status are recorded.
- Cross-validation, comparison, plots and tuning artefact groups are stored.
- The trained pipeline and model metadata are saved for reproducibility.

Notebook tracking evidence

screenshots/mlflow/01_notebook_experiment_tracking_evidence.png

The image shows finished model runs and logged artefact groups. The metric-key verification rows visible in that screenshot are not used as the source for scores; the executed model-comparison outputs above are the metric source.

5. Automated Testing and CI/CD

The GitHub Actions job “Lint, Train, Test, Build and Publish” completed successfully and provides a single auditable delivery path.

Workflow stage	Verification
Install dependencies	Rebuild environment from repository files
Lint code	Ruff quality gate
Train and package	Create model and metadata artefacts
Verify generated files	Confirm expected outputs
Automated tests	Data, preprocessing, model and API tests
Build and run image	Validate container packaging
Endpoint tests	Health, prediction and metrics
Publish	Tag and push to Docker Hub
Upload artefacts	Preserve logs and outputs

SCREENSHOT EVIDENCE: Successful CI/CD workflow

screenshots/github-actions/01_complete_workflow_success.png

The supplied image shows every listed GitHub Actions step with successful status, including Docker Hub publication and workflow artefact upload.

This screenshot must remain in the repository and should be inserted into the submitted PDF if the institution requires image-level proof rather than a screenshot reference.

6. FastAPI Serving and Docker Packaging

FastAPI loads the packaged preprocessing-and-model pipeline and validates JSON input. The API returns a numeric prediction, a readable prediction label and a confidence score.

Endpoint	Function
GET /	Service discovery
GET /health	Runtime/model health
POST /predict	JSON inference and confidence
GET /metrics	Prometheus-compatible metrics
GET /docs	Interactive Swagger documentation

SCREENSHOT EVIDENCE: Swagger prediction

screenshots/api/02_swagger_predict_endpoint.png

POST /predict returned HTTP 200. The supplied response shows prediction 1, label Disease and confidence approximately 0.9236.

SCREENSHOT EVIDENCE: Published Docker image

screenshots/docker/01_dockerhub_published_image.png

The public Docker Hub repository 2025ae05122/heart-disease-api contains latest and commit-derived tags.

The CI workflow builds and runs the image, validates API endpoints, and then publishes the tested image. Minikube pulls the same latest image for deployment.

7. Minikube Deployment and Local API Access

The assignment permits local Kubernetes deployment. The Docker image is deployed with Kubernetes manifests and exposed through a LoadBalancer service. The local tunnel provides a temporary URL for assessor testing.

Evidence item	Observed value
Cluster	Minikube control plane running
Deployment	heart-disease-api; 2/2 ready
Pods	Two pods, each 1/1 Running
Image	docker.io/2025ae05122/heart-disease-api:latest
Service	LoadBalancer, port 80
Access	minikube service heart-disease-api --url

SCREENSHOT EVIDENCE: Deployment, pods and service

screenshots/kubernetes/01_minikube_deployment_pods_service.png

Shows Minikube status, two running pods, the deployed image and LoadBalancer service.

SCREENSHOT EVIDENCE: Deployed prediction endpoint

screenshots/kubernetes/02_deployed_prediction_endpoint.png

The local endpoint returned prediction 0, label No disease and confidence approximately 0.4259.

Assessor access

```
minikube start
kubectl apply -f kubernetes/deployment.yaml
kubectl apply -f kubernetes/service.yaml
minikube service heart-disease-api --url
```

8. Monitoring and Logging

The API emits structured request logs and exports Prometheus metrics. Monitoring is demonstrated with target health and an application-level prediction counter.

Signal	Metric / evidence
API requests	heart_api_requests_total
Request latency	heart_api_request_duration_seconds histogram
Predictions	heart_api_predictions_total counter
Logs	Method, path, status and duration
Scrape health	Prometheus target 1/1 UP

SCREENSHOT EVIDENCE: Kubernetes API logs

screenshots/api/01_kubernetes_api_request_logs.png

Shows recent GET /health requests returning HTTP 200 with measured duration values.

SCREENSHOT EVIDENCE: Prometheus target health

screenshots/monitoring/01_prometheus_target_up.png

Shows heart-disease-api at heart-api:8000/metrics in UP state.

SCREENSHOT EVIDENCE: Prometheus prediction metric

screenshots/monitoring/02_api_metrics_query.png

The query heart_api_predictions_total returns 40 for prediction label 0 at capture time.

Together, the logs and metrics demonstrate runtime request processing, latency measurement, model prediction counting and active metric collection.

9. Conclusion and Submission Deliverables

The project implements the requested MLOps lifecycle from dataset acquisition to a monitored Kubernetes API. Logistic Regression was selected through stratified five-fold cross-validation and achieved held-out ROC-AUC of 0.9610. CI/CD, container publication, local orchestration, endpoint verification, logs and Prometheus metrics provide production-style operating evidence.

Deliverables checklist

Deliverable	Repository location / status
Code and dependencies	src/, api/, requirements.txt, pyproject.toml
Dockerfile	docker/Dockerfile
Cleaned data and downloader	data/processed/ and data/download_dataset.py
Notebook and scripts	notebooks/ and src/train.py
Unit tests	tests/
GitHub Actions	/.github/workflows/ci.yml
Kubernetes manifests	kubernetes/deployment.yaml and service.yaml
Screenshots	screenshots/ category folders
Final report	report/Heart_Disease_MLOps_Final_Report.docx and .pdf
Local API instructions	README: Local Minikube API Access
Code repository	Open GitHub repository
Demonstration video	Add view-only link before final submission
Docker Hub repository	Open Docker Hub repository

Short video recording contents

- Repository and architecture overview.
- Executed notebook: cleaning, EDA, model comparison and final metrics.
- MLflow evidence and packaged artefacts.
- Successful GitHub Actions workflow and Docker Hub image.
- Minikube pods/service and deployed /predict call.
- Kubernetes request logs and Prometheus target/metric query.

Final action before submission: record the short walkthrough, upload it with view-only access, and replace the video placeholder in README and this report with the final link.